

Modulo 2

2026-04-15

R Markdown

This is an R Markdown document. Markdown is a simple formatting syntax for authoring HTML, PDF, and MS Word documents. For more details on using R Markdown see <http://rmarkdown.rstudio.com>.

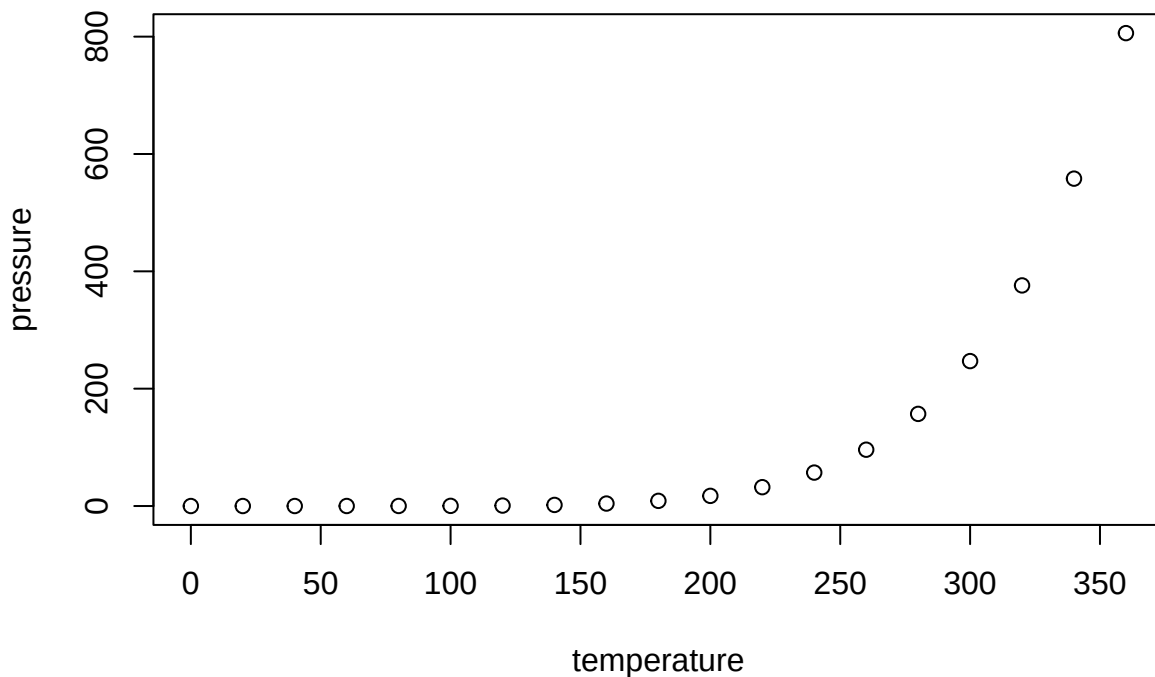
When you click the **Knit** button a document will be generated that includes both content as well as the output of any embedded R code chunks within the document. You can embed an R code chunk like this:

```
summary(cars)
```

```
##      speed      dist
##  Min.   : 4.0    Min.   :  2.00
##  1st Qu.:12.0    1st Qu.: 26.00
##  Median :15.0    Median : 36.00
##  Mean   :15.4    Mean   : 42.98
##  3rd Qu.:19.0    3rd Qu.: 56.00
##  Max.   :25.0    Max.   :120.00
```

Including Plots

You can also embed plots, for example:



Note that the `echo = FALSE` parameter was added to the code chunk to prevent printing of the R code that generated the plot.

Primeros comandos de R

R es un lenguaje muy similar a matlab y permite el trabajo con matrices.

```
a=38
a=40
b=60
c=b-a
c
```

```
## [1] 20
```

```
y=40
alpha<-30
```

Uso de datasets o base de datos internos de R

Usando el comando data en la consola obtenemos distintos datos ya cargados en la base de datos de R, estos datos estan dados en tablas de las cuales podemos elegir columnas especificas si escribimos \$ despues de el comando de informacion especifico y elegimos la columna de datos deseada

```
summary(cars)
```

```
##      speed      dist
##  Min.   : 4.0    Min.   :  2.00
##  1st Qu.:12.0    1st Qu.: 26.00
##  Median :15.0    Median : 36.00
##  Mean   :15.4    Mean    : 42.98
##  3rd Qu.:19.0    3rd Qu.: 56.00
##  Max.   :25.0    Max.    :120.00
```

Comando Plot

Es un comando que permite graficar cualquier fuente de datos que le asignemos, teniendo tambien la posibilidad de asignar nombre a las variables o hacer cambios en el grafico. En caso de duda de como utilizar el comndo podemos escribir ?plot en la consola, lo cual abrira una ventana que explicara en detalle el comando.

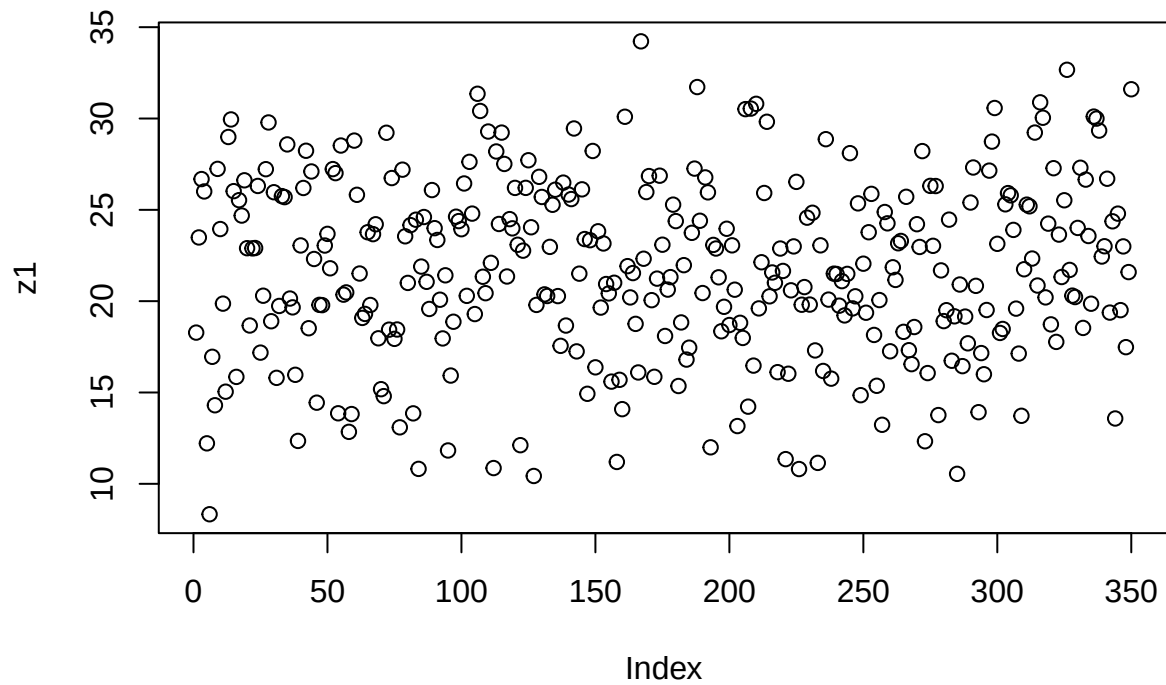
Funciones estadisticas

```
z1<-rnorm(350,22,5)
z1
```

```
## [1] 18.279275 23.487842 26.683385 26.006726 12.215576  8.334561 16.953926
## [8] 14.301118 27.242069 23.948023 19.865022 15.044307 28.988056 29.947828
## [15] 26.022616 15.854716 25.528956 24.687466 26.616270 22.900322 18.668101
## [22] 22.889465 22.908345 26.308093 17.182485 20.298172 27.225047 29.783236
## [29] 18.901067 25.963686 15.801547 19.737800 25.750920 25.695994 28.582548
## [36] 20.143164 19.660926 15.968639 12.348932 23.050080 26.196624 28.234755
## [43] 18.519164 27.099507 22.308679 14.442010 19.797678 19.783069 23.046041
## [50] 23.683995 21.798582 27.230433 27.010026 13.861100 28.520673 20.356565
## [57] 20.479450 12.845377 13.819537 28.791190 25.823293 21.512877 19.082870
## [64] 19.301509 23.772588 19.790512 23.668646 24.209530 17.966662 15.181108
## [71] 14.800358 29.220216 18.440504 26.738985 17.936318 18.448082 13.092278
## [78] 27.199925 23.550972 20.999606 24.162112 13.857388 24.458866 10.816800
## [85] 21.894367 24.606583 21.059124 19.572459 26.083197 23.996940 23.349699
## [92] 20.079864 17.959752 21.415401 11.826806 15.926500 18.873154 24.624778
## [99] 24.383131 23.939281 26.442653 20.290997 27.627340 24.793394 19.293300
```

```
## [106] 31.353970 30.413298 21.336259 20.437397 29.286972 22.098908 10.862155
## [113] 28.194905 24.225724 29.228065 27.511944 21.350604 24.491204 23.985388
## [120] 26.200575 23.088062 12.115161 22.765722 26.202930 27.715133 24.045333
## [127] 10.429388 19.794289 26.806667 25.703885 20.370908 20.282306 22.972166
## [134] 25.276702 26.101291 20.277817 17.553920 26.490166 18.665333 25.836039
## [141] 25.589567 29.454104 17.250462 21.505872 26.122233 23.400572 14.927920
## [148] 23.338179 28.223350 16.377153 23.825537 19.648285 23.158724 20.937960
## [155] 20.423125 15.596034 21.008507 11.197466 15.693354 14.087528 30.096060
## [162] 21.917327 20.204927 21.538480 18.756209 16.097145 34.224182 22.322593
## [169] 25.972730 26.848836 20.054427 15.858799 21.234769 26.869001 23.091839
## [176] 18.094247 20.632735 21.316299 25.283950 24.383766 15.354540 18.834758
## [183] 21.971018 16.801687 17.453677 23.740450 27.257363 31.722191 24.401504
## [190] 20.443846 26.774767 25.956385 11.998006 23.079160 22.882610 21.305063
## [197] 18.350619 19.691289 23.964280 18.694123 23.056686 20.628388 13.161556
## [204] 18.815714 17.986214 30.512958 14.225435 30.542720 16.471429 30.804657
## [211] 19.603234 22.130488 25.921935 29.822093 20.264899 21.571630 21.004470
## [218] 16.112445 22.880672 21.653897 11.352486 16.027150 20.592604 22.997349
## [225] 26.531150 10.808325 19.805489 20.770868 24.570878 19.813826 24.839792
## [232] 17.302642 11.146438 23.056242 16.178528 28.868618 20.084763 15.758183
## [239] 21.505943 21.488435 19.752341 21.096333 19.218755 21.491879 28.102831
## [246] 19.602970 20.266739 25.353340 14.853266 22.052364 19.366252 23.773088
## [253] 25.873564 18.151052 15.366594 20.059914 13.232110 24.880381 24.261015
## [260] 17.255956 21.854577 21.162656 23.170233 23.296329 18.319832 25.713670
## [267] 17.315579 16.552117 18.583793 24.214302 22.966981 28.215634 12.329373
## [274] 16.062358 26.311291 23.030424 26.302523 13.764250 21.683949 18.907708
## [281] 19.506000 24.463113 16.738041 19.170444 10.546545 20.911559 16.442632
## [288] 19.155405 17.691807 25.399617 27.317698 20.838188 13.921048 17.157684
## [295] 15.999085 19.515879 27.147312 28.735266 30.574477 23.138948 18.260096
## [302] 18.478048 25.301450 25.915918 25.792061 23.901457 19.592046 17.129785
## [309] 13.724113 21.747073 25.284450 25.192755 22.333032 29.230759 20.855702
## [316] 30.885275 30.042129 20.206694 24.244762 18.728796 27.280363 17.770097
## [323] 23.644235 21.322085 25.524006 32.667238 21.709846 20.301990 20.224911
## [330] 24.009986 27.303992 18.535407 26.654912 23.566786 19.859635 30.105070
## [337] 29.995315 29.337724 22.445077 23.006431 26.703499 19.374829 24.372033
## [344] 13.577139 24.791538 19.509103 22.997477 17.481784 21.591271 31.601109
```

```
plot(z1)
```



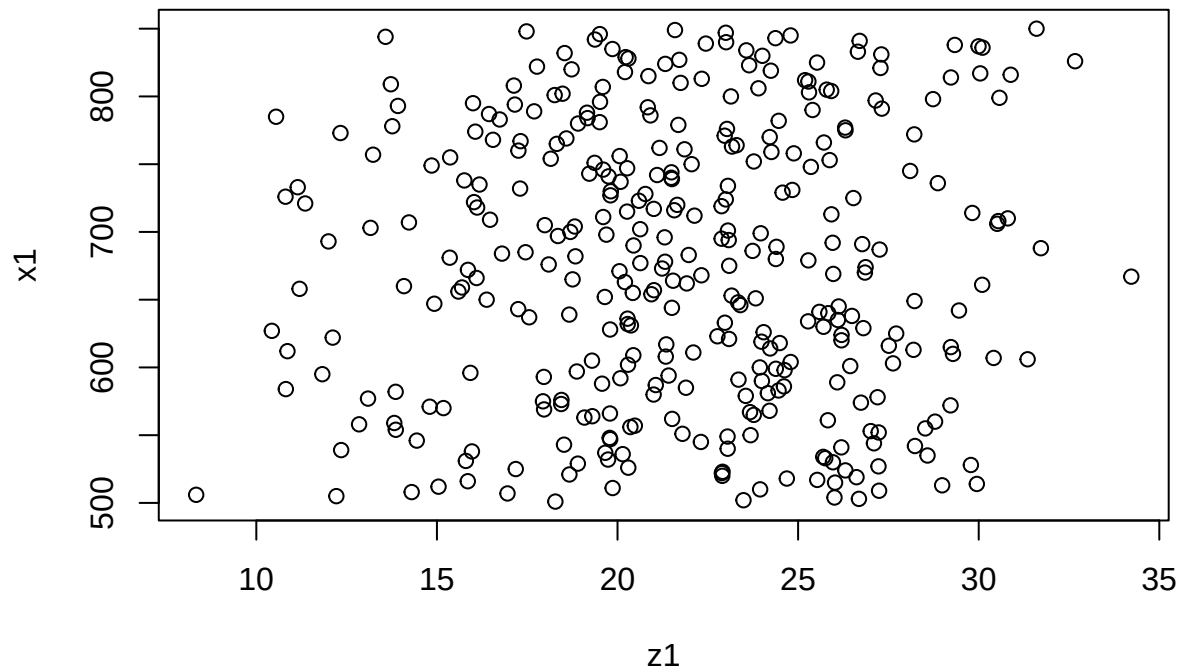
```
w1<-length(z1)
w1
```

```
## [1] 350
```

```
x1<-(501:850)
x1
```

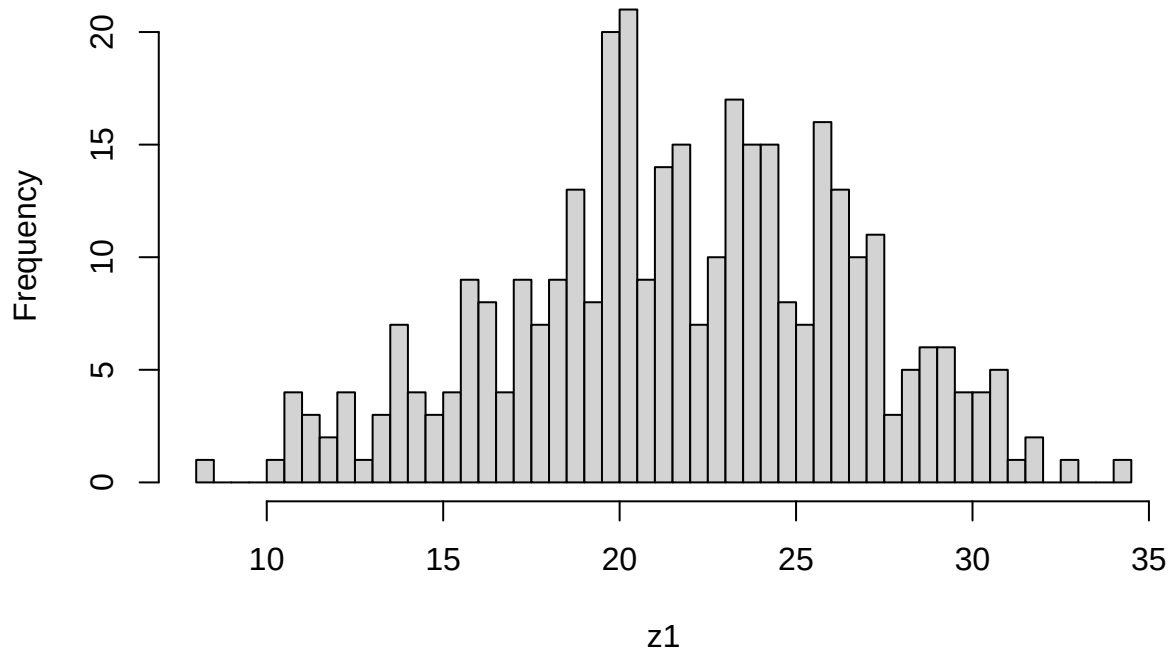
```
## [1] 501 502 503 504 505 506 507 508 509 510 511 512 513 514 515 516 517 518
## [19] 519 520 521 522 523 524 525 526 527 528 529 530 531 532 533 534 535 536
## [37] 537 538 539 540 541 542 543 544 545 546 547 548 549 550 551 552 553 554
## [55] 555 556 557 558 559 560 561 562 563 564 565 566 567 568 569 570 571 572
## [73] 573 574 575 576 577 578 579 580 581 582 583 584 585 586 587 588 589 590
## [91] 591 592 593 594 595 596 597 598 599 600 601 602 603 604 605 606 607 608
## [109] 609 610 611 612 613 614 615 616 617 618 619 620 621 622 623 624 625 626
## [127] 627 628 629 630 631 632 633 634 635 636 637 638 639 640 641 642 643 644
## [145] 645 646 647 648 649 650 651 652 653 654 655 656 657 658 659 660 661 662
## [163] 663 664 665 666 667 668 669 670 671 672 673 674 675 676 677 678 679 680
## [181] 681 682 683 684 685 686 687 688 689 690 691 692 693 694 695 696 697 698
## [199] 699 700 701 702 703 704 705 706 707 708 709 710 711 712 713 714 715 716
## [217] 717 718 719 720 721 722 723 724 725 726 727 728 729 730 731 732 733 734
## [235] 735 736 737 738 739 740 741 742 743 744 745 746 747 748 749 750 751 752
## [253] 753 754 755 756 757 758 759 760 761 762 763 764 765 766 767 768 769 770
## [271] 771 772 773 774 775 776 777 778 779 780 781 782 783 784 785 786 787 788
## [289] 789 790 791 792 793 794 795 796 797 798 799 800 801 802 803 804 805 806
## [307] 807 808 809 810 811 812 813 814 815 816 817 818 819 820 821 822 823 824
## [325] 825 826 827 828 829 830 831 832 833 834 835 836 837 838 839 840 841 842
## [343] 843 844 845 846 847 848 849 850
```

```
plot(z1,x1)
```



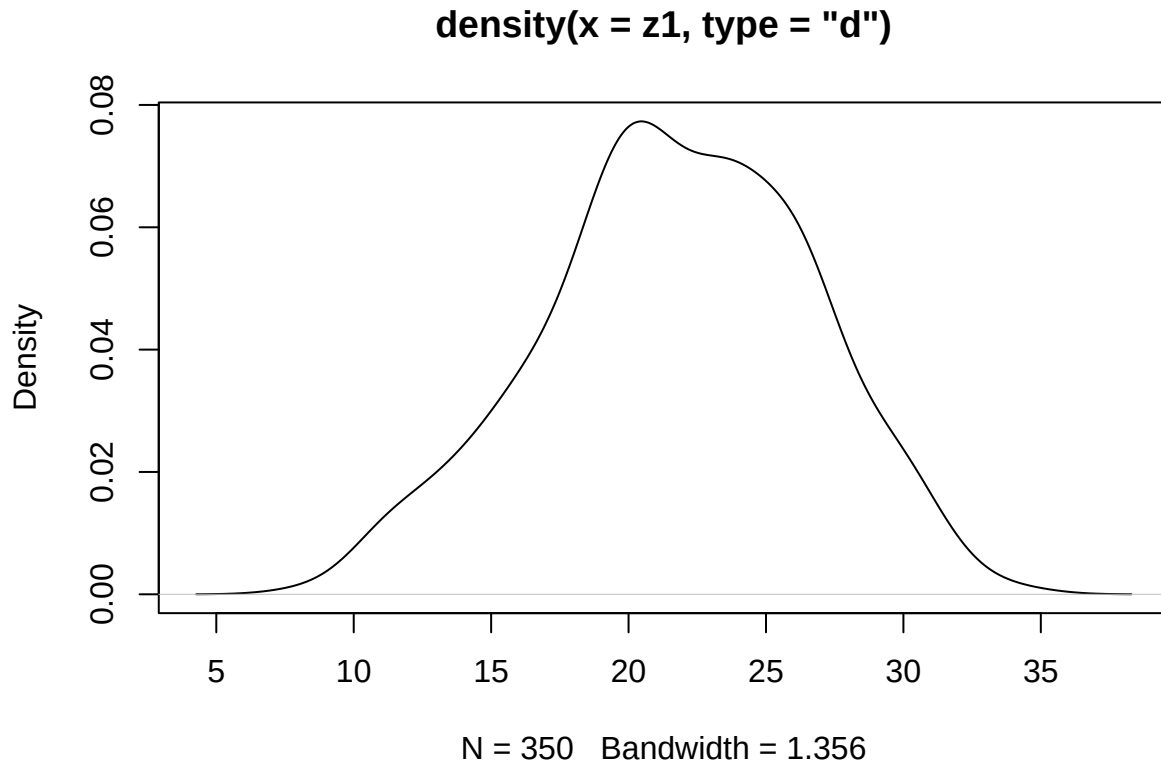
```
hist(z1,main="Histograma de Edades",breaks=50)
```

Histograma de Edades



```
plot(density(z1,type="d"))
```

```
## Warning: In density.default(z1, type = "d") :  
## extra argument 'type' will be disregarded
```



Ejercicio 1

Consigna: las últimas 3 de mi DNI son 341, crear una variable que tenga ese n°.

```
DNI=341
```

Consigna: crear un vector que tenga todos los números enteros desde el 1 hasta el 341.

```
secuencia_dni <- (1:341)
secuencia_dni
```

```
## [1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18
## [19] 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36
## [37] 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54
## [55] 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72
## [73] 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90
## [91] 91 92 93 94 95 96 97 98 99 100 101 102 103 104 105 106 107 108
## [109] 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125 126
## [127] 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143 144
## [145] 145 146 147 148 149 150 151 152 153 154 155 156 157 158 159 160 161 162
## [163] 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179 180
## [181] 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196 197 198
## [199] 199 200 201 202 203 204 205 206 207 208 209 210 211 212 213 214 215 216
## [217] 217 218 219 220 221 222 223 224 225 226 227 228 229 230 231 232 233 234
## [235] 235 236 237 238 239 240 241 242 243 244 245 246 247 248 249 250 251 252
## [253] 253 254 255 256 257 258 259 260 261 262 263 264 265 266 267 268 269 270
## [271] 271 272 273 274 275 276 277 278 279 280 281 282 283 284 285 286 287 288
## [289] 289 290 291 292 293 294 295 296 297 298 299 300 301 302 303 304 305 306
## [307] 307 308 309 310 311 312 313 314 315 316 317 318 319 320 321 322 323 324
## [325] 325 326 327 328 329 330 331 332 333 334 335 336 337 338 339 340 341
```

Ejercicio 2

Calcular la suma de todos los valores del vector `secuencia_dni`

```
total <- 0
valor_final <- length(secuencia_dni)
for (i in 1:valor_final)
total <- total + i
total
```

```
## [1] 58311
```

Ejercicio 3

Consigna: Repetir el ejercicio anterior en Python.

```
suma=0
for i in range(1,341):
    suma+= i
    print(suma)
```

```
## 1
## 3
## 6
## 10
## 15
## 21
## 28
## 36
## 45
## 55
## 66
## 78
## 91
## 105
## 120
## 136
## 153
## 171
## 190
## 210
## 231
## 253
## 276
## 300
## 325
## 351
## 378
## 406
## 435
## 465
## 496
## 528
## 561
## 595
## 630
```

666
703
741
780
820
861
903
946
990
1035
1081
1128
1176
1225
1275
1326
1378
1431
1485
1540
1596
1653
1711
1770
1830
1891
1953
2016
2080
2145
2211
2278
2346
2415
2485
2556
2628
2701
2775
2850
2926
3003
3081
3160
3240
3321
3403
3486
3570
3655
3741
3828
3916
4005

4095
4186
4278
4371
4465
4560
4656
4753
4851
4950
5050
5151
5253
5356
5460
5565
5671
5778
5886
5995
6105
6216
6328
6441
6555
6670
6786
6903
7021
7140
7260
7381
7503
7626
7750
7875
8001
8128
8256
8385
8515
8646
8778
8911
9045
9180
9316
9453
9591
9730
9870
10011
10153
10296

10440
10585
10731
10878
11026
11175
11325
11476
11628
11781
11935
12090
12246
12403
12561
12720
12880
13041
13203
13366
13530
13695
13861
14028
14196
14365
14535
14706
14878
15051
15225
15400
15576
15753
15931
16110
16290
16471
16653
16836
17020
17205
17391
17578
17766
17955
18145
18336
18528
18721
18915
19110
19306
19503

19701
19900
20100
20301
20503
20706
20910
21115
21321
21528
21736
21945
22155
22366
22578
22791
23005
23220
23436
23653
23871
24090
24310
24531
24753
24976
25200
25425
25651
25878
26106
26335
26565
26796
27028
27261
27495
27730
27966
28203
28441
28680
28920
29161
29403
29646
29890
30135
30381
30628
30876
31125
31375
31626

31878
32131
32385
32640
32896
33153
33411
33670
33930
34191
34453
34716
34980
35245
35511
35778
36046
36315
36585
36856
37128
37401
37675
37950
38226
38503
38781
39060
39340
39621
39903
40186
40470
40755
41041
41328
41616
41905
42195
42486
42778
43071
43365
43660
43956
44253
44551
44850
45150
45451
45753
46056
46360
46665

```
## 46971
## 47278
## 47586
## 47895
## 48205
## 48516
## 48828
## 49141
## 49455
## 49770
## 50086
## 50403
## 50721
## 51040
## 51360
## 51681
## 52003
## 52326
## 52650
## 52975
## 53301
## 53628
## 53956
## 54285
## 54615
## 54946
## 55278
## 55611
## 55945
## 56280
## 56616
## 56953
## 57291
## 57630
## 57970
```

Ejercicio 4

¿Cuanto tarda en correr el código del ejercicio 2?

```
inicio <- Sys.time()
total <- 0
valor_final <- 10000000*length(secuencia_dni)
for (i in 1:valor_final)
total <- total + i
total
```

```
## [1] 5.81405e+18
```

```
final <- Sys.time()
final-inicio
```

```
## Time difference of 59.78353 secs
```

Ejercicio 5

Consigna: repetir el ejercicio anterior utilizando la cabeza de Gauss.

```
inicio <- Sys.time()

valor_final <- 10000000 * length(secuencia_dni)

total <- valor_final * (valor_final + 1) / 2

total

## [1] 5.81405e+18
final <- Sys.time()
final - inicio

## Time difference of 0.00208807 secs
```

Ejercicio 6

Consigna: aprende a usar tictoc

```
install.packages("tictoc")

## Installing package into '/cloud/lib/x86_64-pc-linux-gnu-library/4.5'
## (as 'lib' is unspecified)

library(tictoc)
tic("sleeping")
A<-20
print("dormire una siestita...")

## [1] "dormire una siestita..."
## [1] "dormire una siestita..."
Sys.sleep(2)
print("...suena el despertador")

## [1] "...suena el despertador"
## [1] "...suena el despertador"
toc()

## sleeping: 2.005 sec elapsed
```

Ejercicios pdf

Ejercicio 1.4

Consigna: generar un vector secuencia

```
# Metodo 1: bucle for
inicio_for <- Sys.time()

A <- numeric(50000)
for (i in 1:50000) {
  A[i] <- i * 2
}
```

```

final_for <- Sys.time()
tiempo_for <- final_for - inicio_for

# Metodo 2: Generar con SEQ
inicio_seq <- Sys.time()

B <- seq(2, 100000, by = 2)

final_seq <- Sys.time()
tiempo_seq <- final_seq - inicio_seq

# Resultados
tiempo_for

## Time difference of 0.005745649 secs
tiempo_seq

## Time difference of 0.001137972 secs

```

Ejercicios 1.5 y 1.6

Consigna: serie de Fibonacci

```

# Generar Fibonacci hasta superar 1.000.000

fibonacci <- c(0,1)
iteraciones <- 2

while (fibonacci[length(fibonacci)] <= 1000000) {
  nuevo <- fibonacci[length(fibonacci)] + fibonacci[length(fibonacci)-1]
  fibonacci <- c(fibonacci, nuevo)
  iteraciones <- iteraciones + 1
}

fibonacci

## [1]      0      1      1      2      3      5      8     13     21
## [10]     34     55     89    144    233    377    610    987   1597
## [19]   2584   4181   6765  10946  17711  28657  46368  75025 121393
## [28] 196418 317811 514229 832040 1346269

iteraciones

## [1] 32

```

Ejercicio 1.7

Consigna: ordenamiento por método de burbuja

```

install.packages("microbenchmark")

## Installing package into '/cloud/lib/x86_64-pc-linux-gnu-library/4.5'
## (as 'lib' is unspecified)

library(microbenchmark)

# Generar muestra grande

```

```
set.seed(123)
x <- sample(1:100000, 20000)
```

```
# Función burbuja
burbuja <- function(x){
  n <- length(x)
  for (j in 1:(n-1)){
    for (i in 1:(n-j)){
      if (x[i] > x[i+1]){
        temp <- x[i]
        x[i] <- x[i+1]
        x[i+1] <- temp
      }
    }
  }
  return(x)
}
```

```
# Benchmark
resultado <- microbenchmark(
  burbuja = burbuja(x),
  sort_R = sort(x),
  times = 5
)
```

```
resultado
```

```
## Unit: microseconds
##      expr      min       lq      mean     median      uq
## burbuja 20857836.054 21034697.10 2.135478e+07 21532402.76 21664236.54
##  sort_R      623.101      652.27 7.334344e+02      745.54      822.82
##           max neval
## 21684729.379     5
##      823.441     5
```

Ejercicio 1.8

Consigna: Penitencia de Newton

```
# Método 1: algoritmo de sumas parciales; bucle FOR (ineficiente)
inicio1 <- Sys.time()
n <- 10000000
suma1 <- 0
for (i in 1:n) {
  suma1 <- suma1 + i
}

final1 <- Sys.time()
tiempo1 <- final1 - inicio1

# Método 2: Fórmula de Gauss (eficiente)
inicio2 <- Sys.time()

suma2 <- n * (n + 1) / 2
```



```

final2 <- Sys.time()
tiempo2 <- final2 - inicio2

# Resultados
tiempo1

## Time difference of 0.1706605 secs
tiempo2

## Time difference of 0.0008633137 secs

```

Ejercicio Extra

```

library(microbenchmark)
library(ggplot2)

# Definimos burbuja (si no está cargada)
burbuja <- function(x){
  n <- length(x)
  for(j in 1:(n-1)) {
    for(i in 1:(n-j)){
      if(x[i] > x[i+1]){
        temp <- x[i]; x[i] <- x[i+1]; x[i+1] <- temp
      }
    }
  }
  return(x)
}

# Muestra para el gráfico (n = 1000 para que el gráfico sea ágil)
set.seed(123)
datos_v <- sample(1:10000, 1000)

# Medición
bm_ordenacion <- microbenchmark(
  "Burbuja" = burbuja(datos_v),
  "Sort Nativo" = sort(datos_v),
  times = 20
)

# Generar Gráfico de Violín
autoplot(bm_ordenacion) +
  labs(title = "Distribución de Tiempos: Burbuja vs Sort",
        subtitle = "Gráfico de Violín (Microbenchmark)",
        x = "Algoritmo", y = "Tiempo")

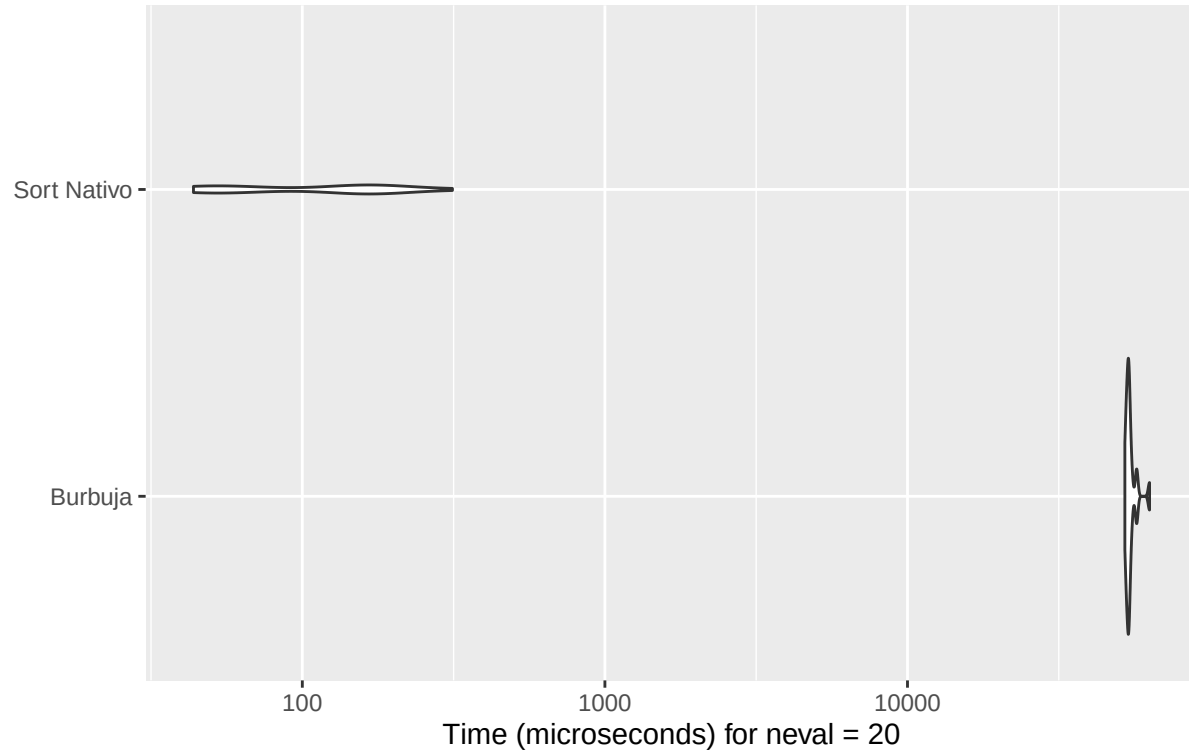
## Warning: `aes_string()` was deprecated in ggplot2 3.0.0.
## i Please use tidy evaluation idioms with `aes()``.
## i See also `vignette("ggplot2-in-packages")` for more information.
## i The deprecated feature was likely used in the microbenchmark package.
## Please report the issue at
## <https://github.com/joshuaulrich/microbenchmark/issues/>.

```

```
## This warning is displayed once per session.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```

Distribución de Tiempos: Burbuja vs Sort

Gráfico de Violín (Microbenchmark)



```
# Simulamos un dataset de 5000 puntos en 2 dimensiones
set.seed(42)
datos_km <- matrix(rnorm(10000), ncol = 2)

# Medimos el tiempo para encontrar 3, 5 y 10 clusters
bm_kmeans <- microbenchmark(
  "K3" = kmeans(datos_km, centers = 3),
  "K5" = kmeans(datos_km, centers = 5),
  "K10" = kmeans(datos_km, centers = 10),
  times = 50
)
```

```
## Warning: did not converge in 10 iterations
```

```
## Warning: Quick-TRANSfer stage steps exceeded maximum (= 250000)
```

```
# Gráfico de Violín para K-means
autoplot(bm_kmeans) +
  labs(title = "Performance de Agrupamiento K-means",
        subtitle = "Comparación por número de clusters (K)",
        x = "Configuración", y = "Tiempo en milisegundos")
```

Performance de Agrupamiento K-means

Comparación por número de clusters (K)

